

* Practical - 4 *

* Write a program to implement Travelling Salesman Problem.

* travelling_salesman.py

```
import itertools
```

```
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0],
]
```

```
city_names = ['A', 'B', 'C', 'D']
num_cities = len(city_names)
```

```
state
```

```
start = 0
```

```
cities = list(range(num_cities))
```

```
min_cost = float('inf')
```

```
best_paths = []
```

```
for perm in itertools.permutations(cities[1:]):
    path = [start] + list(perm) + [start]
    cost = 0
    for i in range(len(path)-1):
```



```
cost += graph[path[i]][path[i+1]]
```

```
name_path = [city_names[i] for i in path]
print(f"Path: {'->'.join(name_path)},
      Cost: {cost}")
```

```
if cost < min_cost:
    min_cost = cost
    best_paths = [path]
elif cost == min_cost:
    best_paths.append(path)
```

```
print("\n Shortest Path(s):")
```

```
for path in best_paths:
    name_path = [city_names[i] for i in path]
    print(f'{'->'.join(name_path)}')
```


* Output *

Path : $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$, Cost: 95

Path : $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$, Cost: 80

Path : $A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$, Cost: 95

Path : $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$, Cost: 80

Path : $A \rightarrow D \rightarrow B \rightarrow C \rightarrow A$, Cost: 95

Path : $A \rightarrow D \rightarrow C \rightarrow B \rightarrow A$, Cost: 95

Shortest Path is :

$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

$A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$